

Simulação de Fluidos SPH usando ECS

Gabriel dos Santos Schmitz
Universidade Tecnológica Federal do Paraná (UTFPR)
Engenharia de Computação, RA: 2487438
✉ gabrielzschmitz@protonmail.com

Resumo—A simulação de fluidos em tempo real exige grande capacidade computacional devido ao cálculo contínuo das interações entre partículas. Entre os métodos utilizados nesse contexto, o *Smoothed Particle Hydrodynamics* (SPH) destaca-se por representar fluidos sem o uso de malhas estruturadas.

Este trabalho apresenta o desenvolvimento de uma simulação de fluidos baseada em SPH integrada à arquitetura *Entity Component System* (ECS). A proposta utiliza o ECS para organizar os dados da aplicação de forma modular, buscando melhorar a estrutura do sistema e a eficiência da execução da simulação.

Palavras chave—Simulação de fluidos, *Smoothed Particle Hydrodynamics*, *Entity Component System*, Computação gráfica, Arquiteturas orientadas a dados

I. INTRODUÇÃO

A simulação computacional de fluidos possui ampla aplicação em áreas como computação gráfica, engenharia, jogos digitais e simulações científicas. Métodos capazes de reproduzir o comportamento dinâmico de líquidos e gases são utilizados para modelar fenômenos físicos complexos, permitindo análises, visualizações e experimentações de forma virtual. Entretanto, a simulação de fluidos em tempo real ainda representa um desafio computacional significativo, principalmente devido à grande quantidade de cálculos necessários para representar interações físicas entre partículas.

Entre os métodos utilizados para simulação de fluidos, destaca-se o *Smoothed Particle Hydrodynamics* (SPH), uma abordagem baseada em partículas originalmente introduzida por Lucy [1] e por Gingold e Monaghan [2]. O método discretiza o fluido em pequenos elementos independentes, denominados partículas, permitindo a aproximação das propriedades físicas do fluido por meio da interação entre seus vizinhos. Diferentemente de métodos baseados em malhas, o SPH apresenta maior flexibilidade para representar superfícies livres, respingos e deformações complexas, tornando-se amplamente utilizado em aplicações interativas e gráficas computacionais.

Na área de computação gráfica, o trabalho *Particle-Based Fluid Simulation for Interactive Applications* [3], de Müller, Charypar e Gross, consolidou o uso do SPH em aplicações interativas ao propor uma abordagem voltada para simulação de fluidos em tempo real. Os autores demonstraram que métodos baseados em partículas podem simplificar o tratamento das equações de conservação de massa e facilitar a representação de superfícies livres, além de permitir a renderização direta das partículas para reconstrução visual do fluido.



Fig. 1. Exemplo de fluido real contido em um recipiente de vidro, ilustrando o comportamento contínuo e deformável característico dos fluidos estudados em simulações computacionais.

Apesar de suas vantagens, implementações de SPH podem apresentar limitações de desempenho devido à necessidade de atualização contínua do estado de milhares de partículas e do cálculo frequente de vizinhança. Nesse contexto, arquiteturas orientadas a dados, como o *Entity Component System* (ECS), surgem como uma alternativa para melhorar organização, modularidade e eficiência computacional. O ECS, popularizado por Scott Bilas na apresentação *A Data-Driven Game Object System* [4], propõe a separação entre dados e comportamento por meio de componentes independentes, favorecendo melhor utilização de memória, maior flexibilidade estrutural e potencial para paralelismo durante a execução.

Este trabalho apresenta o desenvolvimento de uma simulação de fluidos utilizando o método SPH integrado à arquitetura ECS. A implementação foi desenvolvida de forma incremental, iniciando pela modelagem individual de partículas sujeitas a forças básicas, como gravidade e colisão, evoluindo posteriormente para a interação entre múltiplas partículas por meio das equações de Navier-Stokes, formuladas por Claude-Louis Navier e posteriormente desenvolvidas por George Gabriel Stokes [5], amplamente utilizadas na modelagem do movimento de fluidos viscosos [6, 7, 8], e como princípios para o método SPH [9]. O objetivo é investigar como a adoção do ECS pode contribuir para a organização estrutural do sistema e para a execução eficiente das etapas da simulação, mantendo resultados visuais coerentes e estabilidade física.

O restante deste artigo está organizado da seguinte forma: a Seção II descreve os fundamentos teóricos do método SPH e da

arquitetura ECS, bem como os procedimentos incrementais utilizados na implementação da simulação; a Seção III apresenta os resultados obtidos a partir da execução do sistema desenvolvido; a Seção IV analisa os resultados observados, discutindo limitações, impactos e possíveis melhorias da abordagem proposta; por fim, a Seção V resume as principais contribuições do trabalho e apresenta perspectivas para trabalhos futuros.

II. METODOLOGIA

As equações de Navier-Stokes descrevem o comportamento de fluidos em movimento a partir da conservação de massa e da conservação de quantidade de movimento. Para fluidos incompressíveis, essas equações permitem modelar os efeitos de pressão, viscosidade e forças externas sobre o escoamento do fluido.

Embora as equações de Navier-Stokes permitam descrever com precisão o comportamento de fluidos, sua resolução direta na forma contínua pode se tornar computacionalmente custosa, principalmente em aplicações que exigem atualização em tempo real. Isso ocorre porque fluidos são compostos por um número muito grande de partículas interagindo simultaneamente, tornando inviável calcular de forma exata todas as interações físicas presentes no sistema.

Dessa forma, métodos numéricos são utilizados para discretizar o fluido e aproximar seu comportamento de maneira computacionalmente viável. Entre essas abordagens, destacam-se os métodos particulados, nos quais o fluido é representado por um conjunto discreto de partículas interagindo localmente.

Nesse contexto, o método *Smoothed Particle Hydrodynamics* (SPH), originalmente proposto por Lucy [1] e por Gingold e Monaghan [2], tornou-se uma das principais técnicas baseadas em partículas para simulação de fluidos.

No método SPH, cada partícula representa uma pequena porção do fluido e armazena propriedades físicas locais, como massa, densidade, pressão e velocidade. Entretanto, representar diretamente todas as partículas presentes em um fluido real seria computacionalmente inviável, já que fluidos podem ser compostos por milhões de partículas interagindo simultaneamente.

Para tornar a simulação viável computacionalmente, o SPH aproxima o comportamento contínuo do fluido utilizando uma quantidade significativamente menor de partículas. Para isso, o método emprega funções de suavização denominadas *smoothing kernels*, responsáveis por distribuir a influência de cada partícula dentro de uma região limitada do espaço.

Em vez de tratar cada partícula como um ponto isolado, o kernel espalha suas propriedades físicas ao redor de sua vizinhança, permitindo interpolar grandezas físicas de forma contínua entre partículas vizinhas. Na prática, esse processo atua de maneira semelhante a um filtro de suavização (*blur*), reduzindo espaços vazios e discontinuidades entre partículas. Com isso, mesmo utilizando uma quantidade reduzida de partículas, o sistema passa a apresentar comportamento visual e físico semelhante ao de um fluido contínuo.

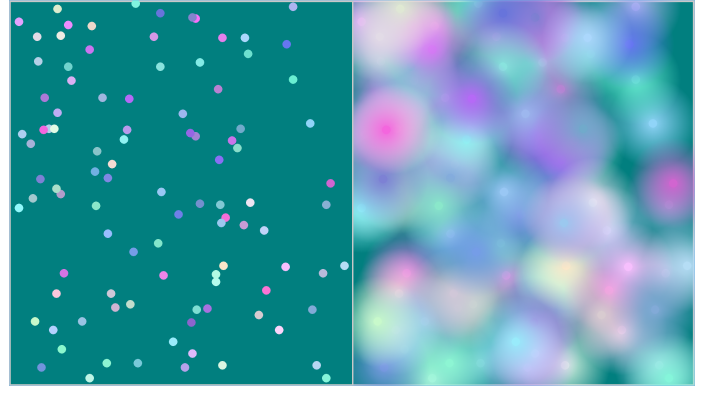


Fig. 2. Mesma distribuição de partículas discretas antes e após a aplicação do kernel de suavização no método SPH.

A partir dessa representação particulada, torna-se possível aproximar as equações da mecânica dos fluidos diretamente sobre o conjunto de partículas. No SPH, as equações de Navier-Stokes são discretizadas de forma que propriedades como pressão, viscosidade e aceleração possam ser calculadas localmente a partir da interação entre partículas vizinhas.

A forma geral da equação de conservação de momento é apresentada na Equação 1:

$$\rho \frac{D\mathbf{u}}{Dt} = -\nabla p + \nabla \cdot \boldsymbol{\tau} + \rho \mathbf{g}, \quad (1)$$

em que $\frac{D}{Dt}$ representa a derivada material, $\boldsymbol{\tau}$ corresponde ao tensor de tensões viscosas, p representa a pressão, ρ a densidade do fluido, $\mathbf{u}$ o campo de velocidades e $\mathbf{g}$ as forças externas aplicadas ao sistema.

Para fluidos Newtonianos incompressíveis com viscosidade constante, o termo viscoso pode ser simplificado conforme a Equação 2:

$$\nabla \cdot \boldsymbol{\tau} = \mu \nabla^2 \mathbf{u}, \quad (2)$$

em que μ representa a viscosidade dinâmica do fluido. Além disso, a derivada material do campo de velocidades corresponde à aceleração do fluido:

$$\frac{D\mathbf{u}}{Dt} = \mathbf{a}. \quad (3)$$

Substituindo as Equações 2 e 3 na Equação 1, obtém-se:

$$\rho \mathbf{a} = -\nabla p + \mu \nabla^2 \mathbf{u} + \rho \mathbf{g}. \quad (4)$$

Dividindo todos os termos pela densidade ρ e definindo a viscosidade cinemática como:

$$\nu = \frac{\mu}{\rho}, \quad (5)$$

obtém-se a forma simplificada das equações de Navier-Stokes utilizada neste trabalho:

$$\mathbf{a} = -\frac{\nabla p}{\rho} + \nu \nabla^2 \mathbf{u} + \mathbf{g}, \quad (6)$$

em que $\mathbf{a}$ representa a aceleração do fluido, p a pressão, ρ a densidade, ν a viscosidade cinemática, $\mathbf{u}$ o campo de velocidades e $\mathbf{g}$ as forças externas aplicadas ao sistema, como a gravidade.

Nessa formulação, o termo do gradiente de pressão acelera o fluido de regiões de maior pressão para regiões de menor pressão. O termo viscoso representa a dissipação interna do fluido, suavizando diferenças de velocidade entre regiões vizinhas, enquanto o termo de forças externas adiciona acelerações provocadas por agentes externos.

Além da conservação de momento, fluidos incompressíveis devem satisfazer a condição de conservação de massa, apresentada na Equação 7:

$$\nabla \cdot \mathbf{u} = 0, \quad (7)$$

indicando que a divergência do campo de velocidades é nula. Isso significa que a quantidade de fluido que entra em uma região é igual à quantidade que sai, mantendo a densidade aproximadamente constante ao longo da simulação.

A. Aplicando à simulação

No método SPH, as equações diferenciais contínuas apresentadas anteriormente (6 e 7) são convertidas em interações discretas entre partículas. Em vez de resolver diretamente as derivadas em um domínio contínuo, o fluido é mapeado por um conjunto finito de nós massivos discretos (as partículas), os quais armazenam propriedades físicas locais, como posição, velocidade, densidade e pressão. A dinâmica macroscópica do meio contínuo emerge, portanto, das interações locais entre partículas vizinhas dentro de um raio de influência delimitado pelo kernel de suavização.

Matematicamente, as grandezas físicas do fluido são aproximadas numericamente sobre esse conjunto discreto. Propriedades contínuas são interpoladas localmente utilizando as contribuições das partículas vizinhas, ponderadas pela função kernel. A aproximação de uma grandeza escalar qualquer A na posição espacial r é regida pela Equação 8:

$$A_S(r) = \sum_j m_j \frac{A_j}{\rho_j} W(r - r_j, h), \quad (8)$$

em que m_j representa a massa da partícula j , A_j denota a grandeza física associada a ela, ρ_j equivale à sua densidade local, r_j define sua posição espacial e $W(r - r_j, h)$ corresponde ao kernel de suavização dotado de raio de suporte h . Dessa forma, os operadores diferenciais presentes na Equação de Navier-Stokes são convertidos em somatórios locais sobre a vizinhança, transformando o sistema contínuo em uma formulação computacionalmente viável.

Para traduzir esse arcabouço matemático em um ambiente virtual de alto desempenho, a implementação do simulador foi estruturada sob o paradigma *Entity Component System* (ECS). Essa abordagem permite a separação estrita entre os dados físicos e os algoritmos de processamento do método SPH. Sob essa ótica, cada partícula é tratada puramente como uma entidade, cujos atributos e estados são distribuídos em componentes especializados, enquanto sistemas independentes encapsulam as etapas de cálculo numérico executadas a cada passo de tempo Δt .

Na arquitetura desenvolvida, a entidade-partícula é composta por quatro estruturas fundamentais:

- **PositionComponent**: armazena as coordenadas espaciais da partícula;
- **VelocityComponent**: mantém o vetor de velocidade linear utilizado na integração temporal;
- **DensityComponent**: encapsula as propriedades de estado do fluido, especificamente a densidade local (ρ) e a pressão interna (p);
- **CircleComponent**: retém os atributos estéticos e primitivas geométricas necessárias para o pipeline de renderização.

O fluxo de processamento físico e a translação desses dados são operados por sistemas dedicados que percorrem as entidades de forma sequencial. Inicialmente, o sistema `ComputeDensity()` calcula a densidade local a partir da distribuição espacial dos vizinhos. Na sequência, os sistemas `ComputePressureForce()` e `ComputeViscosityForce()` determinam, respectivamente, as forças repulsivas de pressão e o termo viscoso associado à dissipação interna de energia.

Dispostas as forças resultantes, o sistema `UpdatePosition()` realiza a integração temporal para atualizar o estado cinemático do sistema. Por fim, o pipeline é concluído com o sistema `RenderParticle()`, responsável pela rasterização do fluido. A Figura 3 ilustra a topologia dessa organização e o fluxo de dependências entre os módulos.

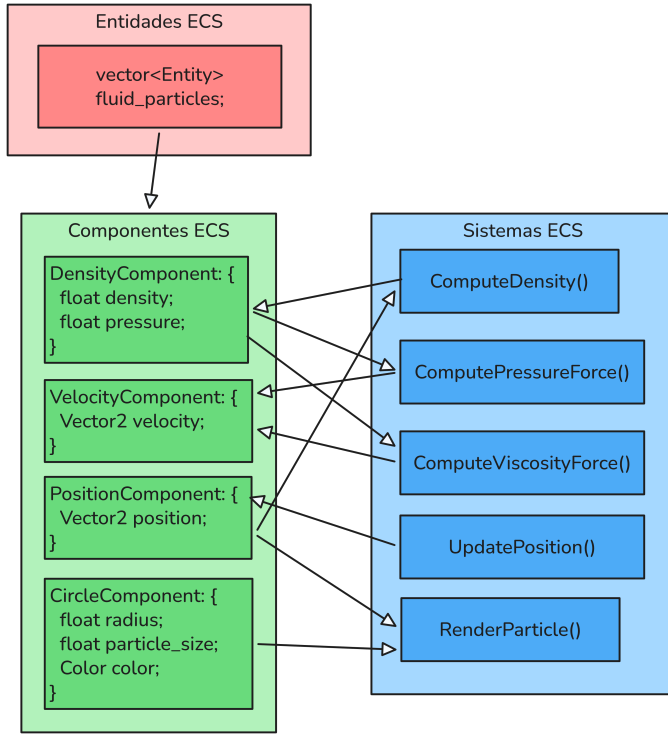


Fig. 3. Organização da simulação SPH utilizando a arquitetura ECS e fluxo de dependências dos sistemas responsáveis pelo cálculo físico, integração temporal e renderização das partículas.

Além de isolar as responsabilidades do código, a adoção da arquitetura ECS oferece vantagens cruciais de desempenho para simulações de mecânica dos fluidos. Como todas as partículas compartilham a mesma composição de componentes, seus dados são alocados de maneira contígua na memória (em arrays densos). Essa organização otimiza a localidade de referência (tanto espacial quanto temporal) e maximiza o aproveitamento das linhas de *cache* do processador, mitigando o gargalo de transferências esparsas de memória (*cache misses*) — um fator determinante para a viabilidade de simulações iterativas contendo milhares de partículas em tempo real.

a.1) Smoothing Kernel

O kernel de suavização define como a influência de uma partícula diminui com a distância. Durante a simulação, cada partícula considera apenas vizinhas localizadas dentro do raio de suporte h , reduzindo o custo computacional e garantindo interações locais.

Para garantir estabilidade e conservação das propriedades físicas do fluido, o kernel deve ser normalizado conforme a Equação 9:

$$\int W(r) dr = 1, \quad (9)$$

garantindo que a soma das contribuições preserve a massa e as propriedades do fluido.

Na prática, o valor do kernel é calculado para cada par de partículas vizinhas, produzindo um peso proporcional à

distância entre elas. Partículas mais próximas exercem maior influência, enquanto partículas distantes possuem contribuição reduzida ou nula.

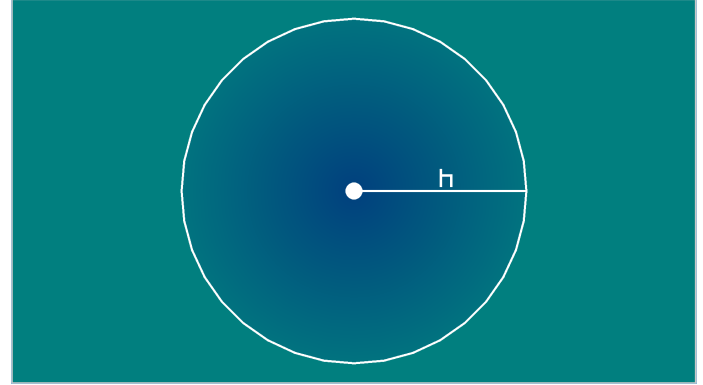


Fig. 4. Representação visual da área de influência (smoothing radius) de uma única partícula central, indicando o decaimento radial da influência do kernel até o raio de suporte h .

a.2) Kernels especializados

A estabilidade, precisão e desempenho do método SPH dependem diretamente da escolha das funções kernel utilizadas durante a simulação. Essas funções são responsáveis por definir como as propriedades físicas de uma partícula influenciam suas vizinhas dentro do raio de suporte h .

Segundo [3], kernels adequados para SPH devem ser normalizados, possuir simetria radial e apresentar valor e derivadas nulas na fronteira do raio de suporte. Essas propriedades contribuem para a estabilidade numérica da simulação e evitam discontinuidades nas forças entre partículas.

Embora o SPH utilize uma formulação geral baseada em kernels de suavização, diferentes operadores físicos exigem propriedades matemáticas específicas. Dessa forma, implementações práticas normalmente utilizam kernels distintos para densidade, pressão e viscosidade.

a) Kernel Poly6

O kernel Poly6 é utilizado principalmente para o cálculo da densidade devido ao seu comportamento suave e estável. Sua formulação é dada por:

$$W_{\rho}(r, h) = \frac{315}{64\pi h^9} \begin{cases} (h^2 - r^2)^3 & 0 \leq r \leq h \\ 0 & \text{caso contrário,} \end{cases} \quad (10)$$

Uma característica importante desse kernel é que a distância aparece apenas na forma quadrática (r^2), permitindo calcular sua influência sem utilizar raízes quadradas durante a computação das distâncias entre partículas. Isso reduz significativamente o custo computacional da simulação.

Além disso, o Poly6 produz distribuições suaves de densidade, tornando-o adequado para a interpolação de grandezas escalares no fluido.

Código 1: Implementação do kernel Poly6

```
float Poly6Kernel(float r2, float h) {
    float h2 = h * h;

    if (r2 > h2)
        return 0.0f;

    float diff = h2 - r2;

    float coeff = 315.0f / (64.0f * PI * pow(h, 9));

    return coeff * pow(diff, 3);
}
```

b) Kernel Spiky

Embora o kernel Poly6 seja eficiente para densidade, ele apresenta problemas no cálculo das forças de pressão. Quando partículas ficam extremamente próximas, o gradiente do Poly6 tende a zero no centro, reduzindo a força de repulsão entre as partículas e favorecendo a formação de agrupamentos artificiais sob altas pressões.

Para evitar esse problema, [3] utiliza o kernel Spiky no cálculo das forças de pressão:

$$W_s(r, h) = \frac{15}{\pi h^6} \begin{cases} (h-r)^3 & 0 \leq r \leq h \\ 0 & \text{caso contrário,} \end{cases} \quad (11)$$

O principal diferencial desse kernel é seu gradiente não nulo próximo ao centro, produzindo forças repulsivas mais intensas entre partículas muito próximas. Isso melhora significativamente a estabilidade do cálculo de pressão e evita aglomerações numéricas no fluido.

Além disso, o kernel Spiky possui derivadas suaves e nulas na fronteira do raio de suporte, reduzindo descontinuidades nas forças calculadas.

Código 2: Implementação do kernel Spiky

```
float SpikyKernel(float r, float h) {
    if (r > h)
        return 0.0f;

    float diff = h - r;

    float coeff = 15.0f / (PI * pow(h, 6));

    return coeff * pow(diff, 3);
}
```

c) Kernel de viscosidade

A viscosidade modela o atrito interno do fluido e possui efeito dissipativo, reduzindo diferenças de velocidade entre partículas vizinhas. Entretanto, kernels tradicionais podem produzir Laplacianos negativos em determinadas regiões, gerando forças viscosas instáveis que aumentam artificialmente a velocidade relativa entre partículas.

Para evitar esse problema, [3] propõe um kernel específico para viscosidade:

$$W_\nu(r, h) = \frac{15}{2\pi h^3} \begin{cases} \left(-\frac{r^3}{2h^3} + \frac{r^2}{h^2} + \frac{h}{2r} - 1\right) & 0 \leq r \leq h \\ 0 & \text{caso contrário,} \end{cases} \quad (12)$$

Esse kernel foi projetado para possuir Laplaciano positivo em todo o domínio, garantindo que as forças viscosas atuem sempre dissipando energia e suavizando o campo de velocidades.

Seu Laplaciano é dado por:

$$\nabla^2 W_\nu(r, h) = \frac{45}{\pi h^6} (h - r), \quad (13)$$

mantendo comportamento estável mesmo em simulações com baixa quantidade de partículas.

A utilização desse kernel melhora significativamente a estabilidade numérica da simulação, reduzindo oscilações e diminuindo a necessidade de técnicas adicionais de amortecimento.

Código 3: Implementação do kernel de viscosidade

```
float ViscosityKernel(float r, float h) {
    if (r <= 0.0f || r > h)
        return 0.0f;

    float r2 = r * r;
    float r3 = r2 * r;

    float h2 = h * h;
    float h3 = h2 * h;

    float coeff = 15.0f / (2.0f * PI * h3);

    float value =
        -(r3 / (2.0f * h3)) +
        (r2 / h2) +
        (h / (2.0f * r)) - 1.0f;

    return coeff * value;
}
```

A Figura 5 compara os três kernels utilizados, juntamente com seus gradientes e Laplacianos.

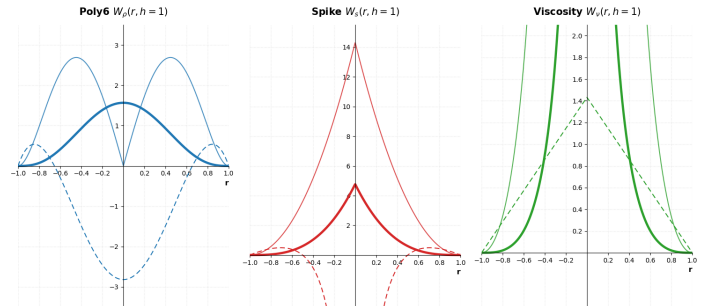


Fig. 5. Os kernels W_ρ , W_s e W_ν com $h = 1$. As linhas espessas representam os kernels, as finas seus gradientes e as tracejadas o Laplaciano, escalado em $0,1\times$.

a.3) Cálculo da densidade

Após determinar as partículas vizinhas, a densidade local de cada partícula é calculada por meio da soma ponderada das

massas vizinhas utilizando o kernel de suavização. Essa etapa corresponde à discretização da conservação de massa.

Cada partícula representa um pequeno volume do fluido definido por:

$$V_i = \frac{m_i}{\rho_i}, \quad (14)$$

em que a massa permanece constante ao longo da simulação, enquanto a densidade varia dinamicamente conforme a distribuição espacial das partículas.

A densidade local é então estimada pela Equação 15:

$$\rho_S(r) = \sum_j m_j W_\rho(r - r_j, h), \quad (15)$$

Esse cálculo está diretamente relacionado à condição de incompressibilidade da Equação 7, pois grandes variações de densidade indicam desvios do comportamento incompressível esperado.

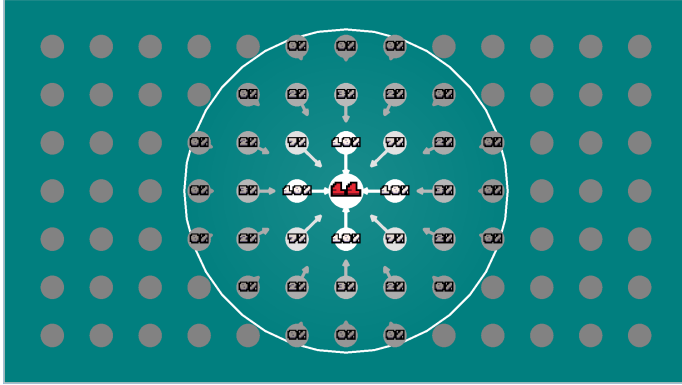


Fig. 6. Representação da visual de densidade local, $\rho_S = 11$, a partir das contribuições ponderadas das partículas vizinhas contidas no raio h .

A implementação da Equação 15 é apresentada no Código 4. Para cada partícula i , o algoritmo percorre suas partículas vizinhas localizadas dentro do raio de suporte h , acumulando suas contribuições por meio da função kernel `Poly6Kernel`. A densidade local é então obtida pela soma ponderada entre a massa m_j das partículas vizinhas e o valor do kernel em função da distância relativa entre as partículas.

Código 4: Cálculo da densidade SPH

```
void ComputeDensity() {
    float h = entities::smoothing_radius;
    float h2 = h * h;
    float m = entities::particle_size;
    size_t N = particle_entities.size();

    for (size_t i = 0; i < N; ++i) {
        Vector2 r = particles[i].position;

        float rho = 0.0f;

        for (size_t j : GetNeighborParticles(i)) {
            Vector2 r_j = particles[j].position;

            float rx = r_j.x - r.x;
```

```
            float ry = r_j.y - r.y;

            float r2 = rx * rx + ry * ry;

            if (r2 <= h2)
                rho += m * Poly6Kernel(r2, h);
        }

        particles[i].density = rho;
    }
}
```

a.4) Cálculo da pressão

Após o cálculo da densidade, a pressão de cada partícula pode ser obtida por uma equação de estado, relacionando a densidade local ρ com a densidade de repouso do fluido. Em métodos SPH fracamente compressíveis, pequenas variações de densidade são utilizadas para gerar forças restauradoras que mantêm o fluido aproximadamente incompressível ao longo da simulação.

Na formulação das equações de Navier-Stokes apresentada na Equação 6, o termo responsável pelas forças de pressão é:

$$-\frac{\nabla p}{\rho}, \quad (16)$$

o qual acelera o fluido das regiões de maior pressão para as regiões de menor pressão. No contexto do SPH, esse termo é discretizado utilizando o gradiente da função kernel.

A aproximação SPH do gradiente de uma grandeza escalar A é dada por:

$$\nabla A_S(r) = \sum_j m_j \frac{A_j}{\rho_j} \nabla W(r - r_j, h), \quad (17)$$

em que m_j representa a massa da partícula vizinha, ρ_j sua densidade e ∇W o gradiente do kernel de suavização.

Aplicando essa formulação ao termo de pressão, obtém-se uma aproximação das forças de pressão entre partículas vizinhas. Essas forças produzem acelerações repulsivas em regiões comprimidas do fluido, afastando partículas muito próximas e contribuindo para a manutenção da estabilidade numérica da simulação.

Para esse cálculo, é utilizado o kernel `Spiky`, apresentado anteriormente na Equação 11. Diferentemente do kernel `Poly6`, o `Spiky` possui gradiente não nulo próximo ao centro, produzindo forças repulsivas mais intensas entre partículas muito próximas e evitando aglomerações artificiais no fluido.

A Figura 7 ilustra o efeito das forças de pressão entre partículas, responsáveis por acelerar o fluido em direção às regiões de menor pressão.

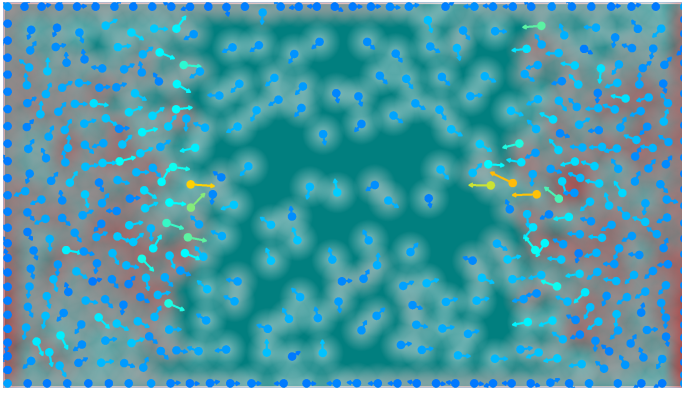


Fig. 7. Representação das forças de pressão entre partículas vizinhas, produzindo aceleração em direção às regiões de menor pressão. Na simulação, as áreas brancas representam a pressão alvo do fluido, enquanto regiões em vermelho indicam alta pressão e regiões em azul representam baixa pressão.

A implementação do cálculo das forças de pressão é apresentada no Código 5. Para cada partícula i , o algoritmo percorre suas partículas vizinhas localizadas dentro do raio de suporte h , acumulando suas contribuições por meio do gradiente da função kernel `SpikyKernelGradient`. A força de pressão é então obtida a partir da pressão p_i e da densidade ρ_j das partículas vizinhas, produzindo forças repulsivas proporcionais à compressão local do fluido e orientadas na direção entre as partículas.

Código 5: Cálculo das forças de pressão SPH

```
void ComputePressureForce() {
    float h = entities::smoothing_radius;
    float m = entities::particle_size;
    size_t N = particle_entities.size();

    for (size_t i = 0; i < N; ++i) {
        Vector2 r = particles[i].position;
        float rho_i = particles[i].density;
        float p_i = particles[i].pressure;

        Vector2 pressure_force = {0.0f, 0.0f};
        for (size_t j : GetNeighborParticles(i)) {
            if (i == j) continue;

            Vector2 r_j = particles[j].position;
            float rho_j = particles[j].density;
            float p_j = particles[j].pressure;

            float rx = r_j.x - r.x;
            float ry = r_j.y - r.y;

            float dist = sqrt(rx * rx + ry * ry);
            if (dist > 0.0f && dist <= h) {
                Vector2 grad =
                    SpikyKernelGradient(rx, ry, dist, h);

                float pressure =
                    -m * (p_i + p_j) / (2.0f * rho_j);

                pressure_force.x += pressure * grad.x;
                pressure_force.y += pressure * grad.y;
            }
        }
        particles[i].pressure_force = pressure_force;
    }
}
```

a.5) Cálculo da viscosidade

A viscosidade é responsável pela dissipação de energia e pelo amortecimento das diferenças de velocidade entre partículas vizinhas. Esse efeito modela o atrito interno do fluido e contribui para a estabilidade da simulação.

No SPH, o termo viscoso é obtido utilizando o Laplaciano do kernel de suavização. O operador Laplaciano é aproximado pela Equação 18:

$$\nabla^2 A_S(r) = \sum_j m_j \frac{A_j}{\rho_j} \nabla^2 W(r - r_j, h). \quad (18)$$

Essa aproximação permite discretizar diretamente o termo viscoso das Equações de Navier-Stokes:

$$\nu \nabla^2 \mathbf{u}, \quad (19)$$

em que ν representa a viscosidade cinemática do fluido.

Na prática, esse termo suaviza diferenças bruscas de velocidade entre partículas próximas, reduzindo oscilações numéricas e produzindo um movimento mais contínuo e estável.

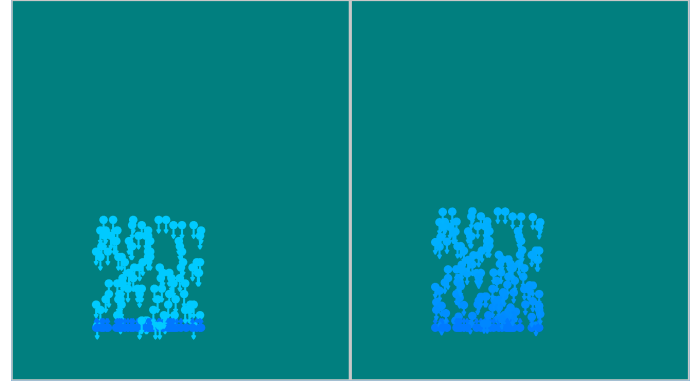


Fig. 8. Representação do efeito viscoso no método SPH. A viscosidade suaviza diferenças locais de velocidade entre partículas vizinhas, produzindo um escoamento mais contínuo e estável.

A implementação do termo viscoso é apresentada no Código 6. Para cada partícula i , o algoritmo percorre suas partículas vizinhas localizadas dentro do raio de suporte h , calculando a diferença de velocidade entre as partículas e aplicando o Laplaciano do kernel de viscosidade por meio da função `ViscosityKernelLaplacian`. A contribuição viscosa é obtida a partir da viscosidade cinemática ν , da massa m , da densidade da partícula vizinha ρ_j e da diferença relativa de velocidades. Essas contribuições são acumuladas no vetor `viscosity_force`, produzindo uma força que suaviza diferenças bruscas de velocidade entre partículas próximas, aproximando numericamente o termo viscoso ($\nu \nabla^2 \mathbf{u}$) das equações de Navier-Stokes.

Código 6: Cálculo das forças viscosas no método SPH

```
void ComputeViscosityForce() {
    float h = entities::smoothing_radius;
    float m = entities::particle_size;
```

```

float nu = entities::viscosity;
size_t N = particle_entities.size();

for (size_t i = 0; i < N; ++i) {
    Vector2 v_i = particles[i].velocity;
    float rho_i = particles[i].density;

    Vector2 viscosity_force = {0.0f, 0.0f};
    for (size_t j : GetNeighborParticles(i)) {
        if (i == j) continue;

        Vector2 v_j = particles[j].velocity;
        float rho_j = particles[j].density;
        Vector2 r =
            particles[j].position
            - particles[i].position;

        float dist = sqrt(r.x * r.x + r.y * r.y);
        if (dist > 0.0f && dist <= h) {
            float laplacian =
                ViscosityKernelLaplacian(dist, h);

            Vector2 velocity_diff = {
                v_j.x - v_i.x,
                v_j.y - v_i.y
            };

            float viscosity =
                (nu * m * laplacian) / rho_j;

            viscosity_force.x +=
                viscosity * velocity_diff.x;
            viscosity_force.y +=
                viscosity * velocity_diff.y;
        }
    }

    particles[i].viscosity_force = viscosity_force;
}
}

```

Após o cálculo das forças de pressão, viscosidade e forças externas como a gravidade g , a aceleração total de cada partícula é atualizada. Com isso, velocidade e posição podem ser integradas numericamente ao longo do tempo, permitindo a evolução dinâmica do fluido.

Dessa forma, as equações contínuas de Navier-Stokes passam a ser representadas por interações locais entre partículas vizinhas, permitindo simular o comportamento macroscópico do fluido a partir de operações discretas e computacionalmente viáveis.

III. RESULTADOS

Esta seção apresenta os resultados obtidos com a implementação do método *Smoothed Particle Hydrodynamics* (SPH) utilizando a arquitetura *Entity Component System* (ECS). Inicialmente, são apresentadas as configurações de hardware, software e os parâmetros utilizados nos experimentos. Em seguida, é analisado o desempenho da implementação em função da quantidade de partículas, considerando a taxa de quadros por segundo e o tempo de processamento de cada passo da simulação.

A. Configuração da simulação

Os experimentos foram realizados utilizando uma simulação bidimensional baseada em partículas SPH. Cada partícula

representa uma pequena porção do fluido e armazena propriedades físicas locais utilizadas durante os cálculos de densidade, pressão e viscosidade.

A implementação foi desenvolvida e executada em um notebook Acer Predator Helios Neo 16S AI. A Tabela 1 apresenta a configuração de hardware e software utilizada durante os experimentos.

Tabela 1. Configuração do ambiente utilizado nos experimentos.

Componente	Especificação
Sistema operacional	CachyOS Linux x86_64
Kernel	Linux 7.0.10
Processador	Intel Core Ultra 9 275HX
Quantidade de núcleos	24 núcleos, 24 threads
Frequência máxima	5.40 GHz
GPU dedicada	NVIDIA GeForce RTX 5070 Ti Mobile
GPU integrada	Intel Graphics
Memória RAM	32 GB DDR5 6400MHz

Os experimentos foram realizados variando a quantidade de partículas entre 50 e 10,000. Os demais parâmetros foram mantidos constantes durante as execuções. A Tabela 2 apresenta os valores utilizados.

Tabela 2. Parâmetros utilizados na simulação SPH.

Parâmetro	Valor
Quantidade de partículas	50–10000
Tamanho de partícula	2.0
Raio de suporte (h)	50.0
Passo temporal (Δt)	0.2
Passos simulados (N_s)	250
Densidade alvo (ρ)	0.000425
Viscosidade (ν)	0.8
Multiplicador de Pressão ($-\nabla p/\rho$)	250.0
Gravidade (g)	1.0
Resolução da simulação	1920 × 1080

B. Resultados visuais

As Figuras 9a e 9b apresentam diferentes momentos da execução da simulação SPH. As imagens permitem visualizar o comportamento das partículas durante a execução do programa, evidenciando a formação e o deslocamento do fluido no ambiente de simulação.

C. Desempenho computacional

O desempenho foi avaliado em duas configurações. A primeira corresponde à execução *headless*, na qual a simulação é executada sem a etapa de renderização gráfica. A segunda corresponde à execução completa, incluindo a renderização das partículas na resolução de (1920×1080).

Essa distinção permite avaliar separadamente o custo associado ao processamento da simulação e o custo adicional introduzido pela renderização. Nos gráficos apresentados a seguir, a série representada em amarelo corresponde à execução *headless*, enquanto a série representada em vermelho corresponde à execução com renderização.

c.1) Taxa de quadros

A Figura 10 apresenta a taxa de quadros por segundo (FPS) em função da quantidade de partículas para as duas configurações de execução.

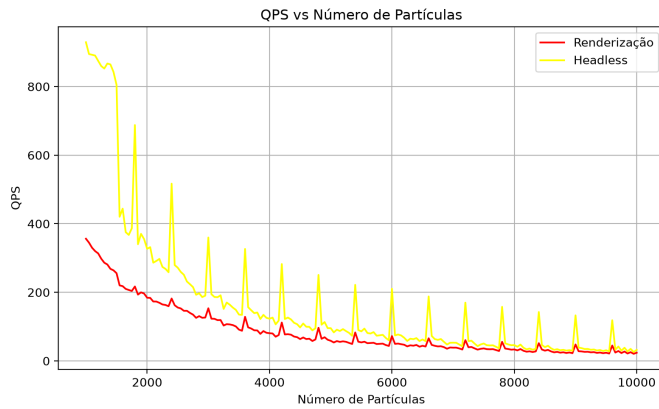


Fig. 10. Taxa de quadros por segundo em função da quantidade de partículas. A série amarela corresponde à execução *headless* e a série vermelha à execução com renderização.

Em ambas as configurações, observa-se uma redução da taxa de quadros à medida que a quantidade de partículas aumenta. Esse comportamento decorre do aumento do número de operações necessárias para calcular as propriedades e interações entre as partículas.

A execução *headless* apresenta desempenho superior à execução com renderização em toda a faixa avaliada. Para 10.000 partículas, por exemplo, a execução *headless* atingiu aproximadamente (32,2) FPS, enquanto a execução com renderização atingiu aproximadamente (23,9) FPS. Portanto, nessa configuração, a renderização introduziu um custo adicional perceptível sobre o desempenho total da aplicação.

c.2) Tempo de processamento

A Figura 11 apresenta o tempo médio necessário para processar cada passo da simulação em função da quantidade de partículas.

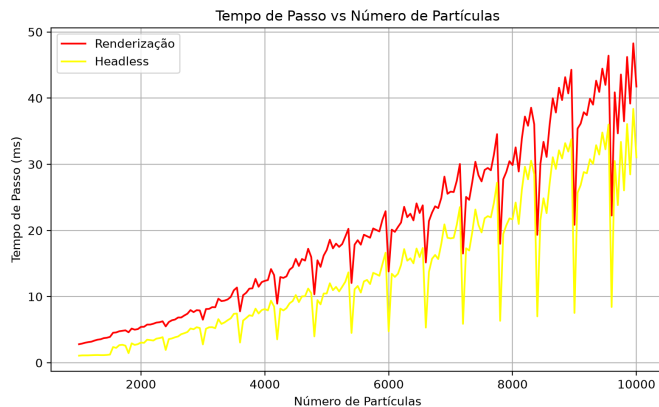


Fig. 11. Tempo de processamento por passo em função da quantidade de partículas. A série amarela corresponde à execução *headless* e a série vermelha à execução com renderização.

Os resultados apresentam o comportamento inverso ao observado no gráfico de FPS: o tempo necessário para executar cada passo aumenta com a quantidade de partículas. Para 10.000 partículas, o tempo médio por passo foi de aproximadamente (31,0) ms na execução *headless* e (41,8) ms na execução com renderização.

A diferença entre as duas configurações evidencia o impacto da renderização sobre o tempo total de execução. Enquanto a versão *headless* permite avaliar predominantemente o custo computacional da simulação SPH, a versão com renderização inclui também o processamento necessário para apresentar os resultados graficamente.

Considerando os dois indicadores em conjunto, observa-se que o aumento da quantidade de partículas reduz progressivamente o desempenho da aplicação. Ainda assim, a versão com renderização mantém aproximadamente (24) FPS com 10.000 partículas, enquanto a versão *headless* permanece acima de (30) FPS nessa configuração.

IV. DISCUSSÃO

A. Limitações

A principal limitação observada está relacionada ao custo computacional das interações entre partículas, que aumenta conforme a quantidade de partículas é ampliada. Esse comportamento limita a escalabilidade da implementação e reduz a taxa de quadros para configurações com maior número de partículas.

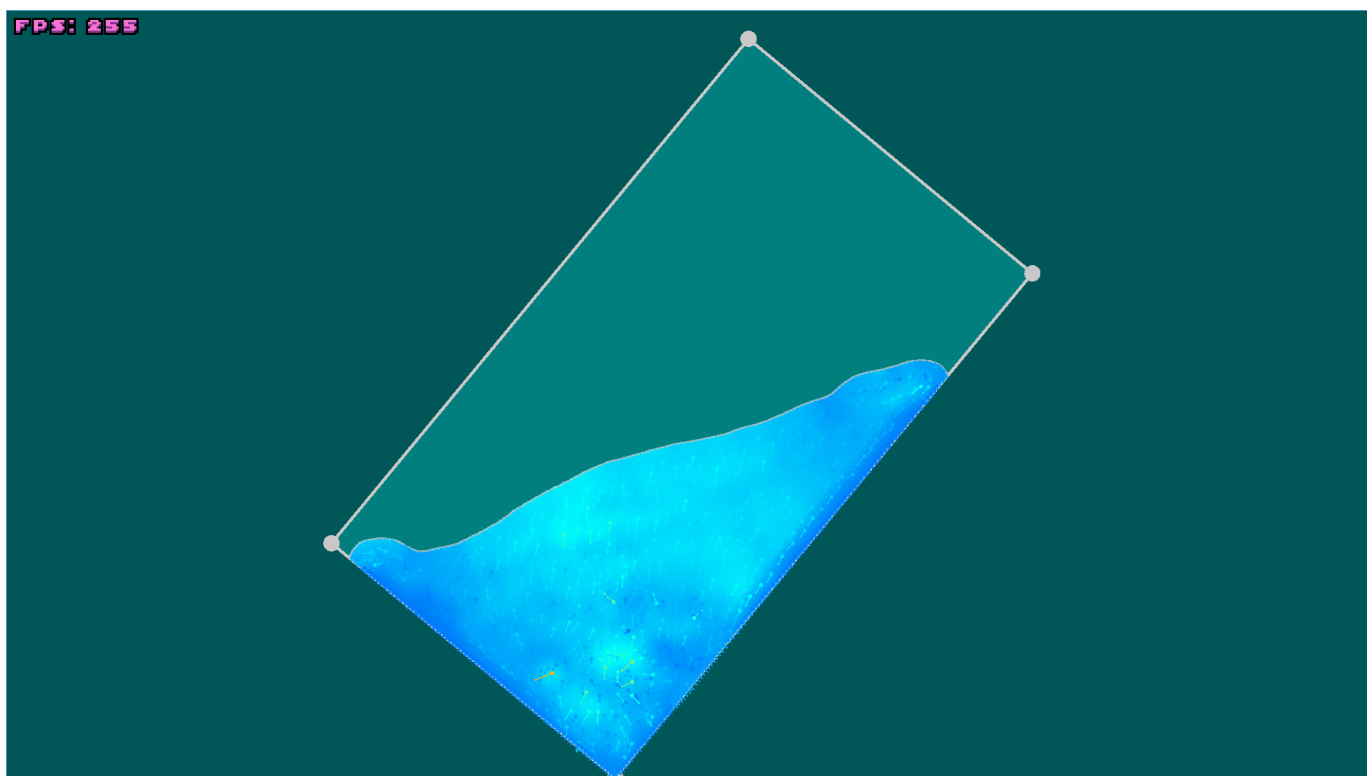
Outra limitação está relacionada ao custo adicional da renderização. Embora a versão *headless* apresente desempenho superior, a execução completa precisa realizar tanto os cálculos da simulação quanto a representação gráfica das partículas.

O programa também utiliza uma implementação falha de multi-thread. O que explica os picos negativos em fps e tempo de passo, parte da simulação tem problemas tentando acessar dados em outras threads resultando em perda abrupta de dados de algumas partículas.

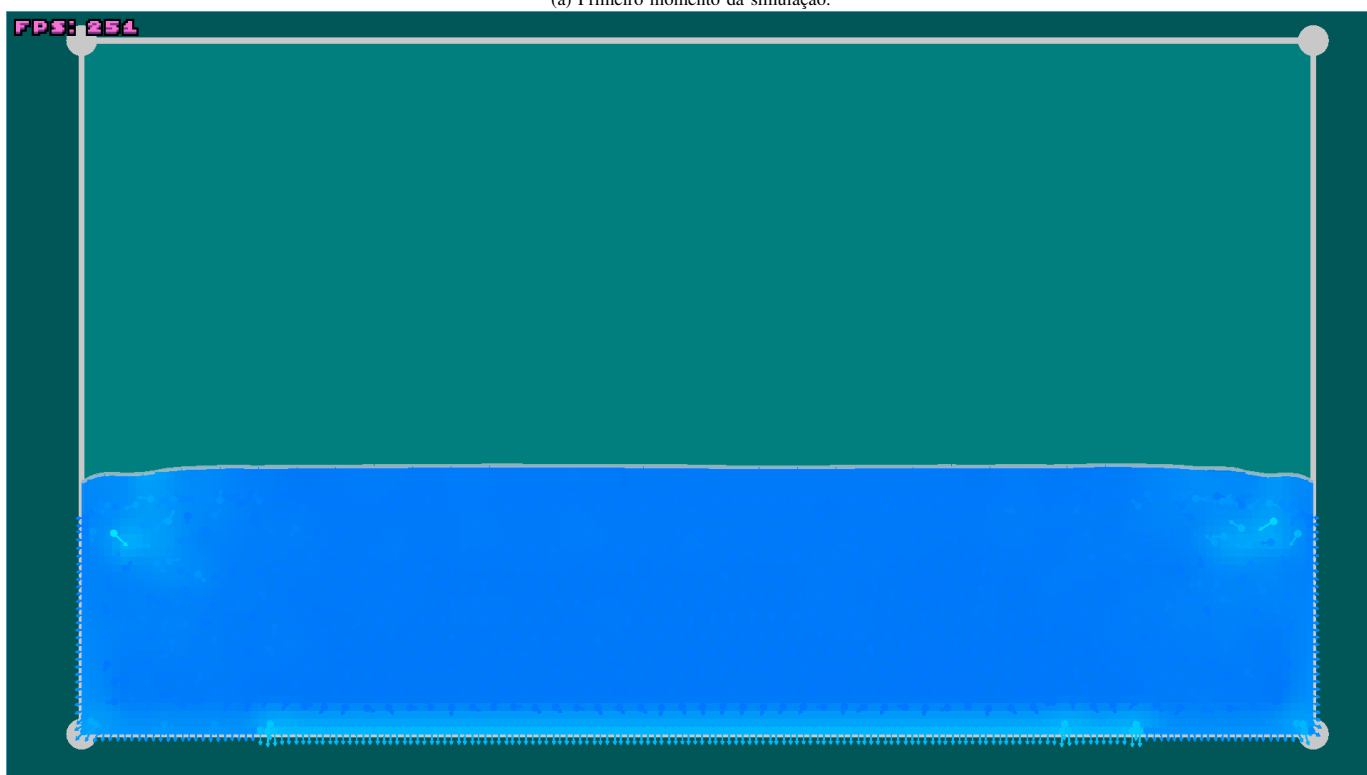
Além disso, não foram incorporadas técnicas de otimização presentes em simuladores mais avançados, como aceleração em GPU, estruturas espaciais especializadas para busca de vizinhos e refinamento adaptativo. A utilização dessas técnicas poderia reduzir o custo das interações e permitir a execução de simulações com maior quantidade de partículas mantendo uma taxa de quadros mais elevada.

V. CONCLUSÃO

Apresente um resumo das principais descobertas do estudo, reforçando sua relevância e impacto. Evite introduzir novas informações nesta seção.



(a) Primeiro momento da simulação.



(b) Segundo momento da simulação.

Fig. 9. Diferentes momentos da execução da simulação SPH.

REFERÊNCIAS

- [1] L. B. Lucy. “A Numerical Approach to the Testing of the Fission Hypothesis”. Em: *The Astronomical Journal* 82.12 (1977), pp. 1013–1024. URL: <https://articles.adsabs.harvard.edu/pdf/1977AJ.....82.1013L>.
- [2] R. A. Gingold e J. J. Monaghan. “Smoothed Particle Hydrodynamics: Theory and Application to Non-Spherical Stars”. Em: *Monthly Notices of the Royal Astronomical Society* 181 (1977), pp. 375–389. URL: <https://academic.oup.com/mnras/article/181/3/375/988212>.
- [3] Matthias Müller, David Charypar e Markus Gross. “Particle-Based Fluid Simulation for Interactive Applications”. Em: *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. Eurographics Association, 2003, pp. 154–159. DOI: 10.2312/SCA03/154-159. URL: <https://matthias-research.github.io/pages/publications/sca03.pdf>.
- [4] Scott Bilas. *A Data-Driven Game Object System*. Game Developers Conference (GDC). Presentation on the architecture used in Dungeon Siege. 2002. URL: <https://www.gamedevs.org/uploads/data-driven-game-object-system.pdf>.
- [5] Morris Kline. *Mathematical Thought from Ancient to Modern Times*. Oxford University Press, 1972. ISBN: 0-19-506136-5.
- [6] G. K. Batchelor. *An Introduction to Fluid Dynamics*. Cambridge, UK: Cambridge University Press, 1967. DOI: 10.1017/CBO9780511800955.
- [7] L. D. Landau e E. M. Lifshitz. *Fluid Mechanics*. Course of Theoretical Physics, Vol. 6. Oxford, UK: Pergamon Press, 1987. DOI: 10.1016/B978-0-08-033933-7.50009-X.
- [8] Joel H. Ferziger e Milovan Peric. *Computational Methods for Fluid Dynamics*. 3ª ed. Berlin, Germany: Springer, 2002. DOI: 10.1007/978-3-642-56026-2.
- [9] J. J. Monaghan. “Smoothed Particle Hydrodynamics”. Em: *Reports on Progress in Physics* 68.8 (2005), pp. 1703–1759. DOI: 10.1088/0034-4885/68/8/R01.